

Two phases, in sequence

The corpus consolidation finishes first. AESOP XI's build-out starts once it does. This is the plan as confirmed tonight — nothing here has started except Phase 1.

SETTLED TERMINOLOGY

- harness** one specific, swappable execution environment — Prime Agent is a harness, Claude Code+ECC is a harness. Has an engine (does the work) and its own rules/state.
- Æsop-Xi** the orchestrator / protocol layer (repo: aesop-xi-protocol) — "the tactical, narrative logic." Decides which harness runs which task, and is the constant standard both are held to. Deliberately *not* a harness itself — that's what lets it outlive any one harness being swapped out.

OFFICIAL SIGNAGE — FROM NOVÆGENTI DEFINED (PT.1)

LAYER	REPO	BRANDING
Presentation	horizons-ui	Horizons UI™ — the ever-expanding canopy
Execution	novaecopia-core	NovÆcopia — the abundance of the system
Protocol	aesop-xi-protocol	Æsop-Xi — the tactical, narrative logic
Data Core	nova-corpus	NovÆ-Corpus — the trunk/body of knowledge

Tree metaphor: nova-corpus is the Trunk, aesop-xi-protocol + novaecopia-core are the Roots, horizons-ui is the Leaves and Sky.

Ægenti is the singular/unified brand form (per the source document, "NovusÆxenti" was an explicitly rejected variant — "NovÆgenti" is the one chosen). **Æxentis** is a separate, deliberate form used in plural/collective constructions like the Drive folder `___Lex-Novi-Æxentis-Copia` ("Lex Novi Æxentis Copia") — not an error, confirmed by the user. This document previously used "AESOP XI" and "NovA-Corpus" without the Æ ligature throughout — not yet corrected everywhere in this session's other material. Repo/URL/dev-facing names stay plain ASCII always (novaecopia, aesop-xi, nova-corpus) — the Æ forms are UI branding only.

PHASE 01 Corpus consolidation

in progress – the only phase currently running

Not an open question: the xgrep pass, keyword search, and document separation/organizing is model-directed action, full stop.

The actual open question is mechanics only – once local processing tools are installed for the heavy compute, does the model run documents through itself directly, or does it author an automation script that performs the work? These aren't competing paths to choose between – same outcome either way, most likely blended.

DIRECT

Model processes each document through itself, interactively – the approach used throughout tonight's session.

SCRIPTED

Model authors an automation script; the script performs the bulk heavy compute while the model directs and reviews.

– not either/or, likely both –

target output per file: four-document schema (original + structured markdown + clean text + JSONL, minus whichever the source already is) → filed into the vault layout, where each component gets its own Wiki + CLAUDE.md + skills folder, and action-description files become skills.

PHASE 02 AESOP XI build-out

planned – starts after Phase 1 completes

Five steps. Step 3 branches into its own session; step 5 rejoins the branch that paused at step 2.

1

Wire the harness stack. SONNET-TIER ECC, Prime Agent, mem0, OB1, OmniRoute, reverse-skill, Honey, and the rest – wired for real, checked for actual collisions (MCP server names, ports), not the theoretical pass done tonight.

2

Run Pocock's skill sequence. OPUS-TIER 5–6 skills including implement, for repo scaffolding – through scope and strategy definition only. Stops before implementation starts.

3

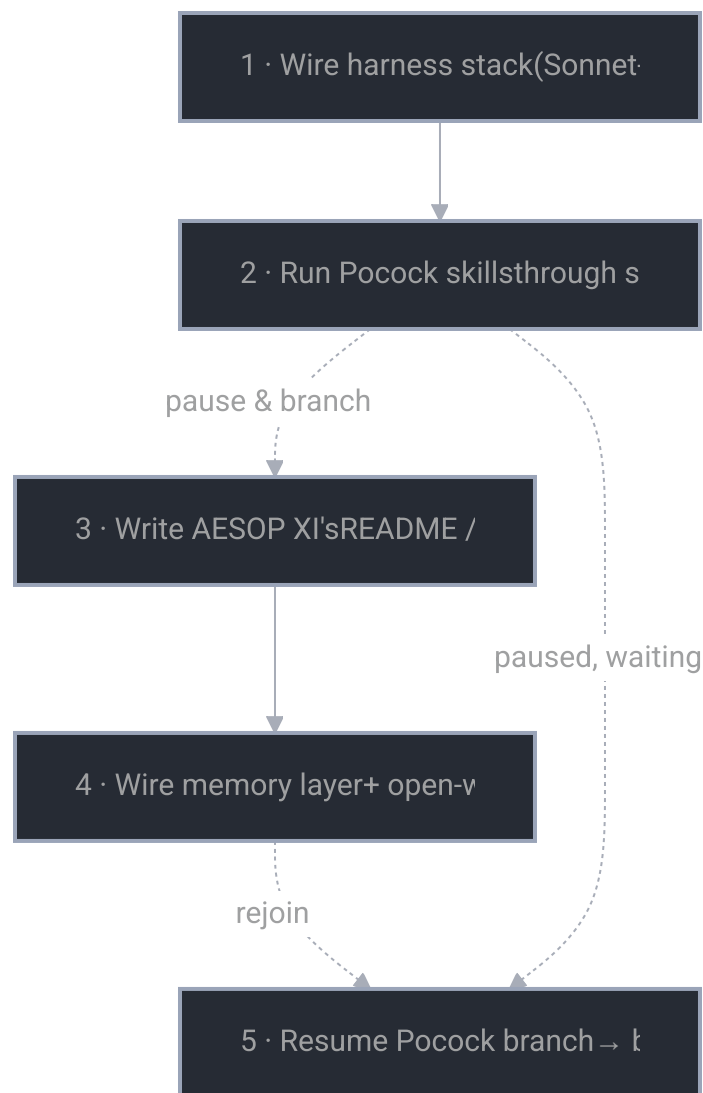
Branch off here. A separate session picks up from the stop point: write AESOP XI's README / whitepaper / mission statement, finish hard-wiring its protocols, decide which harnesses are swappable per environment or node.

4

Wire the memory layer. Once the universal/enterprise memory bank format *and* the protocol/orchestration layer are both defined — full wiring, then the open-wiki LLM-wiki agent logs and snapshots it as the persistent memory layer underneath everything going forward.

5

Rejoin and build. Back to the paused Pocock-skills branch from step 2. An orchestrating model runs the actual multi-repo build.



step 2 pauses and waits — steps 3-4 run in a separate session before step 5 resumes it

target output: 5 repos — Horizons UI (orchestrator: WebSocket layer, Chromium

WebView, LLM chat tile, terminal GUI, model router + fallback OpenRouter, houses the nano/smol-agent driving NPU query/execute) + 2 daemon APKs it depends on – each a bare-metal, low-graphics GUI (1-2 tiles, for local config and portability, not headless) – (shell-access daemon via Accessibility Service, bypassing the Termux sandbox; speech/vision daemon, same permission class) – not three independent apps, one orchestrator plus two services it can't function without – plus one repo for AESOP XI itself, one for the Obsidian Vault / memory bank.

LAUNCH PROMPT – PASTE INTO A NEW SESSION ONCE PHASE 1 IS DONE

Resume the AESOP XI build-out (Phase 2). Read the roadmap artifact and the 2026-08-14 handoff in NovA-Corpus/ for full context. Step 1: wire the real harness stack (ECC, Prime Agent, mem0, OB1, OmniRoute, reverse-skill, Honey, and the rest already inventoried) and check for actual MCP-server-name and port collisions – not a theoretical pass, verify each one live. Step 2, only after step 1 is clean: run Matt Pocock's skill sequence through scope and strategy definition only – implement is in scope to invoke, but stop before it starts implementing. Do not proceed past that stop point without confirming with the user first.